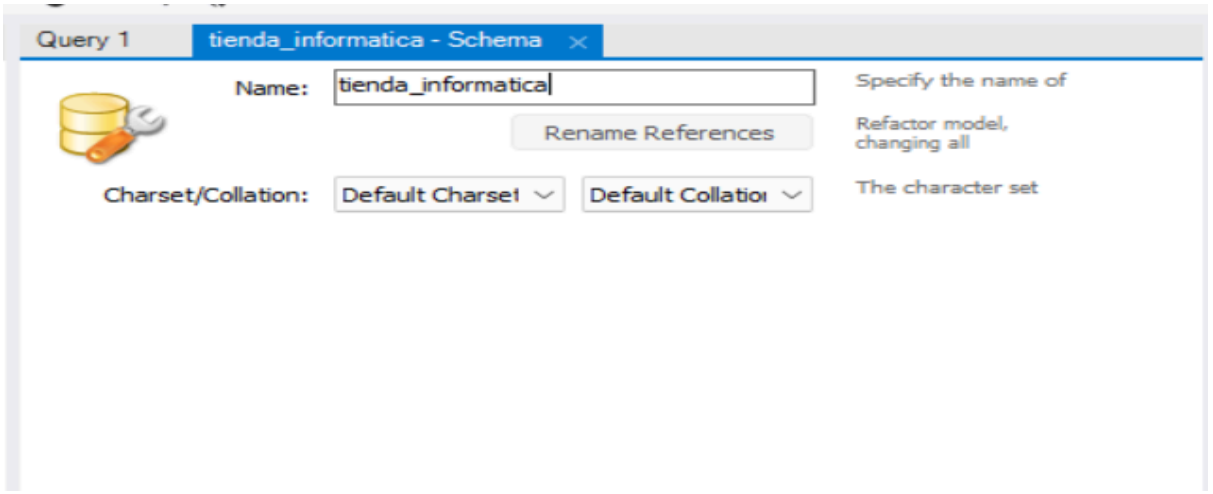
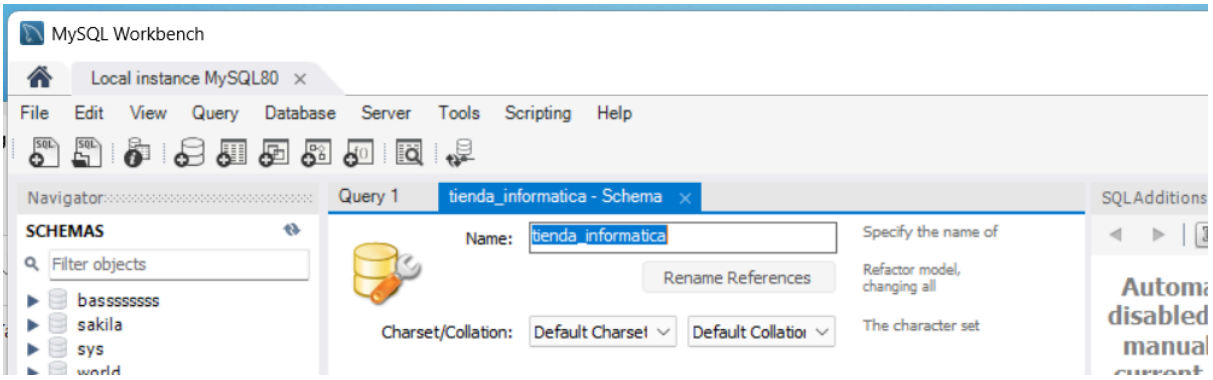
CREACIÓN DE MI BASE DE DATO TIENDA_INFORMATICA.

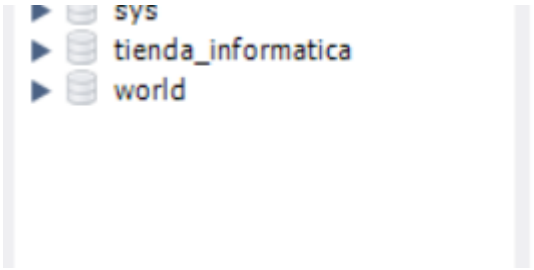
T1. creación de mi base de dato (tienda_informatica).

estoy usando mysql workbench.





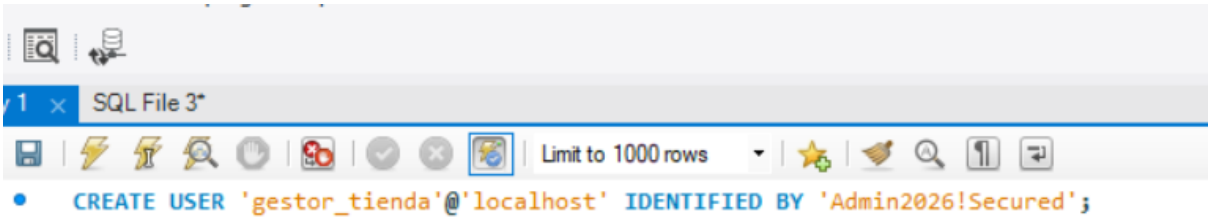
ya está creado mi base de dato (tienda_informatica)



CREACIÓN DE LOS USUARIO Y SUS IDENTIFICACION.

1. Crea un usuario llamado **gestor_tienda** que únicamente pueda conectarse desde la máquina local con la contraseña segura (explica qué requisitos estableces para que la contraseña sea segura).

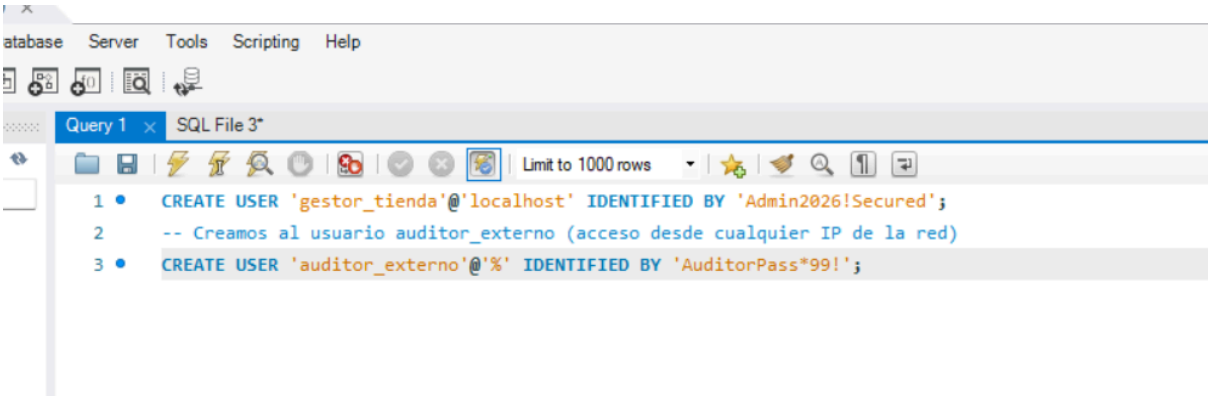
CREATE USER 'gestor_tienda'@'localhost' IDENTIFIED BY 'Admin2026!Secured';



- **Usuario creado:** 'gestor_tiendar'
- **Host autorizado:** localhostt' solo puede entrar si está físicamente en el servidor o conecta localmente.
- **Contraseña asignada:** 'Admin2026!Secured';

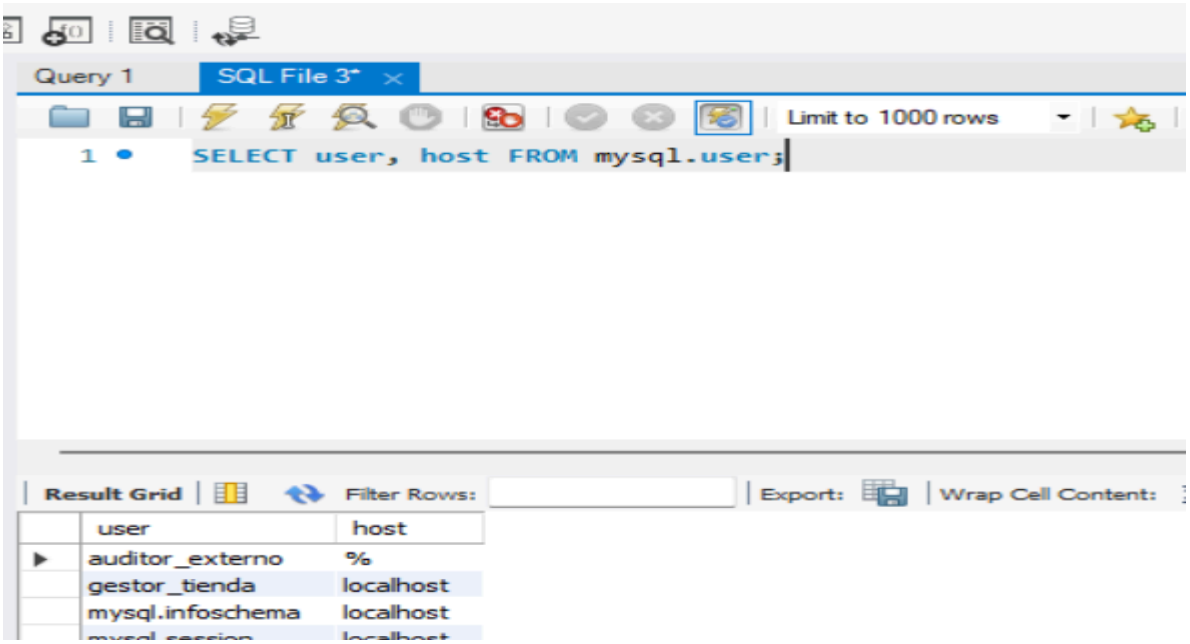
2. Creando el usuario llamado auditor_externo que pueda conectarse desde cualquier dirección IP de la red.

CREATE USER 'auditor_externo'@'%' IDENTIFIED BY 'AuditorPass*99!';



verificar si están creados con el comando.

SELECT user, host FROM mysql.user;

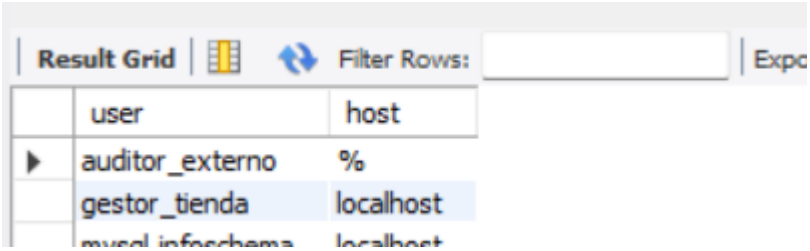


- **Usuario creado:** auditor_externo'
- **Host autorizado:** localhostt' solo puede entrar si está físicamente en el servidor o conecta localmente.
- **Contraseña asignad**a: 'AuditorPass*99!';

PARA VERIFICAR SI LOS USUARIOS ESTÁN CREADOS.

Ejecuta el siguiente comando para ver si ambos usuarios están bien creados.

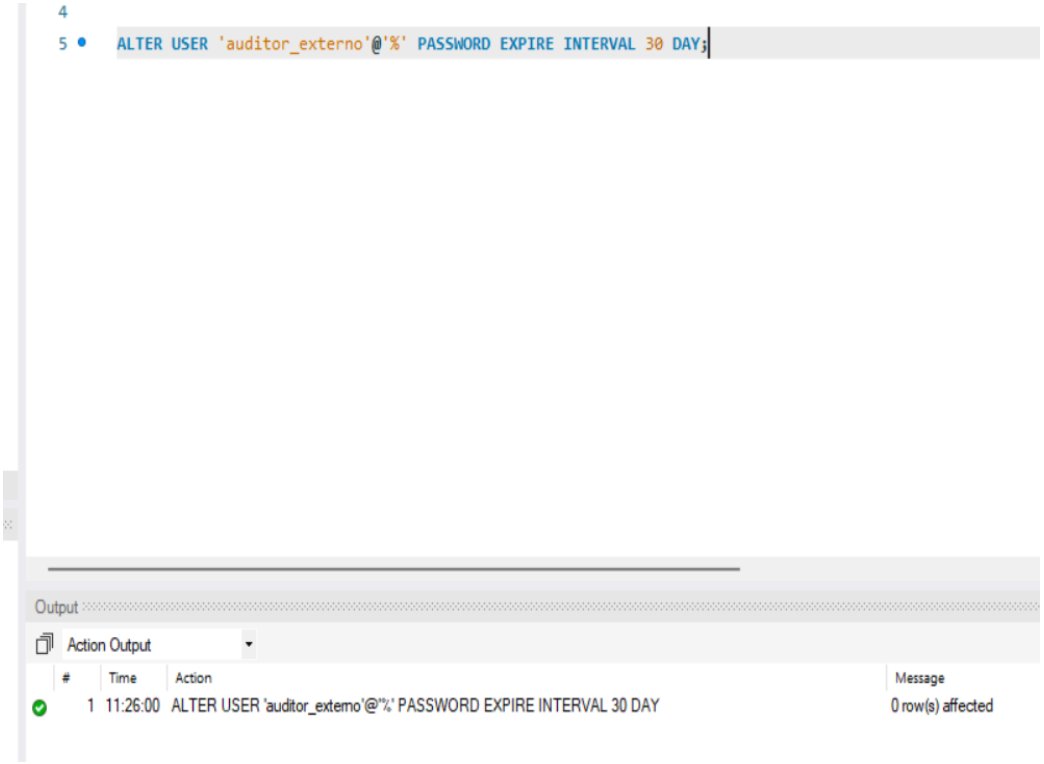
```
SELECT user, host FROM mysql.user;
```



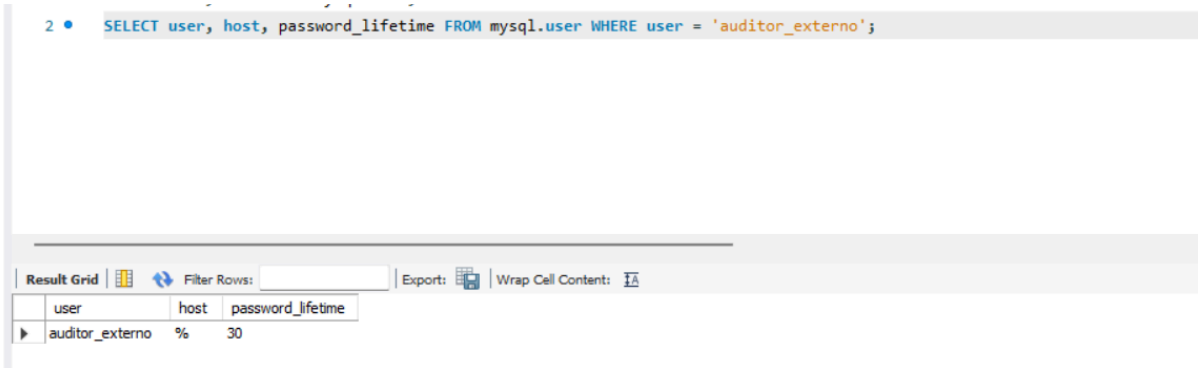
CONFIGURAR LA DIRECTIVA DE CADUCIDAD Y SESIÓN.

1. Directiva de caducidad de credenciales: Configura el usuario auditor_externo para que su contraseña caduque obligatoriamente cada 30 días, obligándole.

ALTER USER 'auditor_externo'@'%' PASSWORD EXPIRE INTERVAL 30 DAY;



comprobandolo con el comando
SELECT user, host, password_lifetime FROM mysql.user WHERE user = 'auditor_externo';



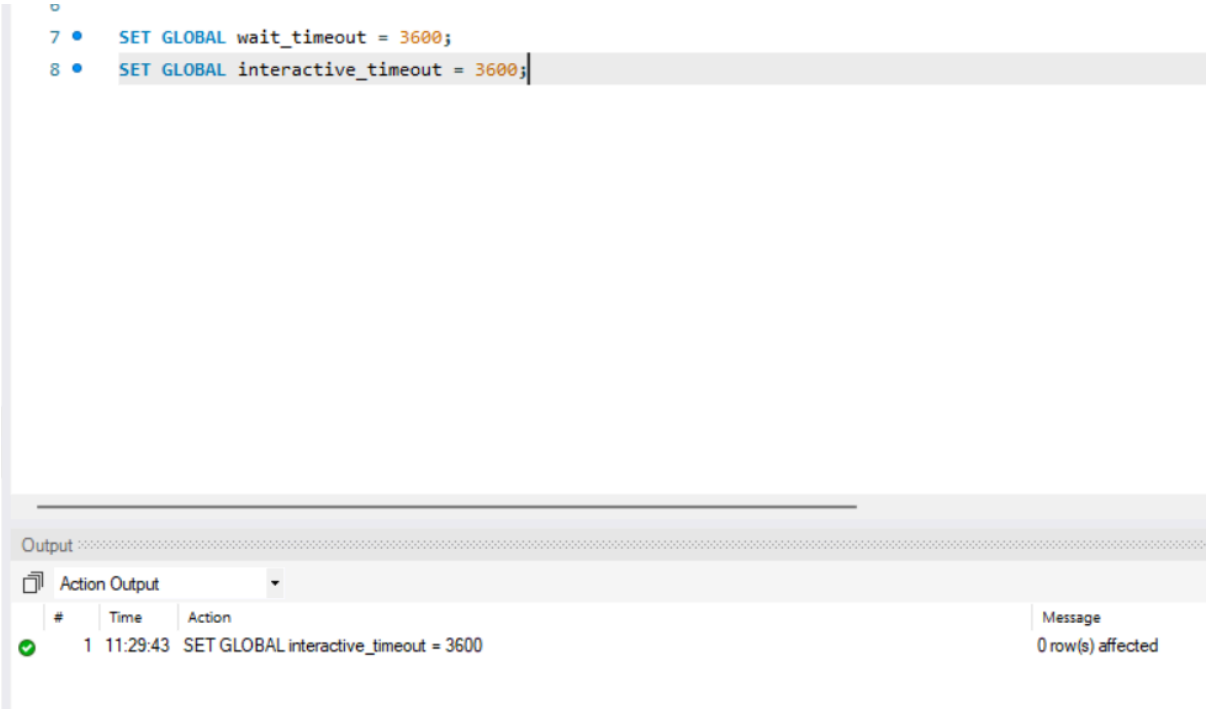
- Lo que estoy haciendo aquí es modificar la cuenta del 'auditor_externo para que obliga el usuario cambiar su contraseña en 30 día o se caduca.
- Es por tema de seguridad

CONTROL DE TIEMPO O SESSION.

1. **Directiva de control de sesión:** Establece una restricción técnica para que las conexiones activas del usuario gestor_tienda no puedan superar 1 hora de inactividad o tiempo de vida activo, forzando la reautenticación tras dicho periodo.

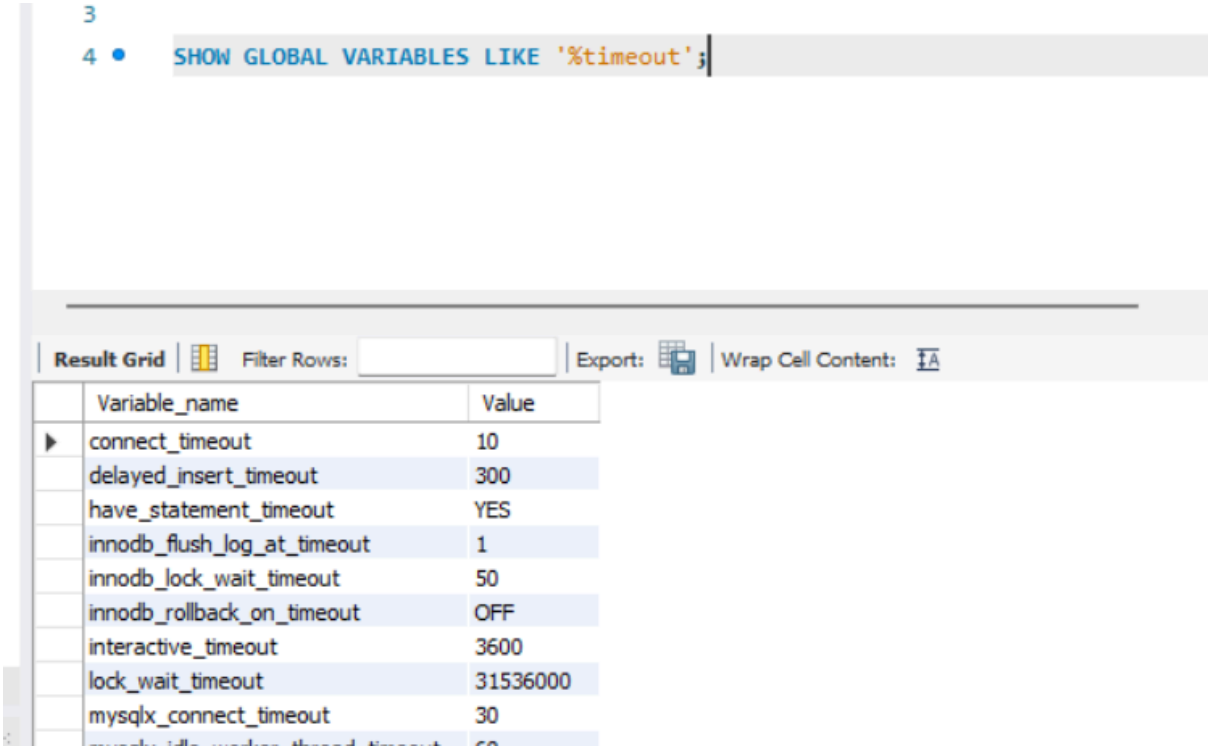
Establecer el tiempo límite de inactividad a 1 hora (3600 segundos)

```
SET GLOBAL wait_timeout = 3600;
SET GLOBAL interactive_timeout = 3600;
```



verificarlo con el comando .

```
SHOW GLOBAL VARIABLES LIKE '%timeout';
```



- Sirve para ordenarle a la base de datos que cierre automáticamente cualquier conexión que lleve más de 1 hora (3600 segundos) sin hacer nada.

T2. CREACIÓN Y GESTIÓN DE ROLES.

1. Definición de rol operativo: Crea un rol denominado rol_consultor.

CREATE ROLE 'rol_consultor';

```
9
10 •   GRANT 'rol_consultor' TO 'auditor_externo'@'%';
11
12 •   DROP ROLE IF EXISTS 'rol_consultor';
13 •   CREATE ROLE 'rol_consultor';
14 •   GRANT SELECT ON tienda_informatica.* TO 'rol_consultor';
15 •   GRANT 'rol_consultor' TO 'auditor_externo'@'%';
16 •   SET DEFAULT ROLE 'rol_consultor' TO 'auditor_externo'@'%';
```

verificando si el rol creado existe
SELECT user, host FROM mysql.user WHERE user = 'rol_consultor';

```
5
6 •   SELECT user, host FROM mysql.user WHERE user = 'rol_consultor';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Result Grid

user	host
rol_consultor	%

- **`CREATE ROLE 'rol_consultor';`**: Crea un "grupo" de permisos (un rol) llamado `rol_consultor` al que luego le podremos asociar usuarios.

2. **Concesión de privilegios al rol**: Otorga al rol rol_consultor permisos de solo lectura sobre toda la base de datos tienda_informatica.

   GRANT SELECT ON tienda_informatica.* TO 'rol_consultor';

```
9
10 •  GRANT 'rol_consultor' TO 'auditor_externo'@'%';
11
12 •  DROP ROLE IF EXISTS 'rol_consultor';
13 •  CREATE ROLE 'rol_consultor';
14 •  GRANT SELECT ON tienda_informatica.* TO 'rol_consultor';
15 •  GRANT 'rol_consultor' TO 'auditor_externo'@'%';
16 •  SET DEFAULT ROLE 'rol_consultor' TO 'auditor_externo'@'%';
```

verificar si el rol esta asignado con el permiso de leer (select)
SELECT * FROM mysql.default_roles;

```
7 •  SELECT * FROM mysql.default_roles;
```

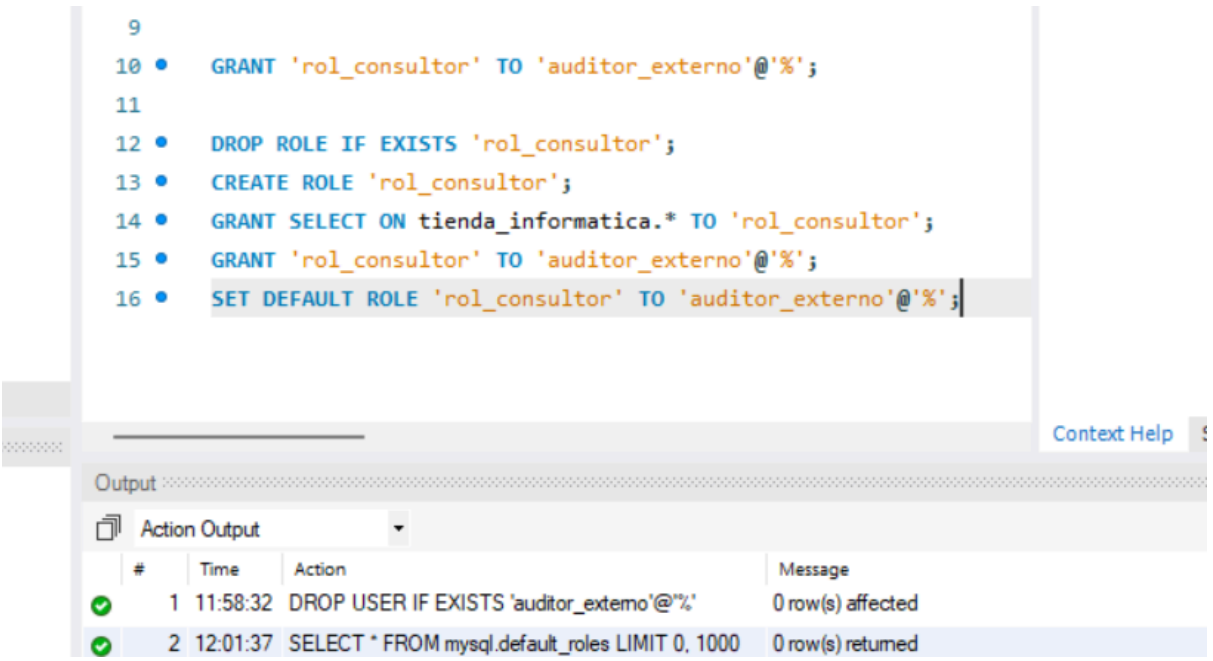
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap C

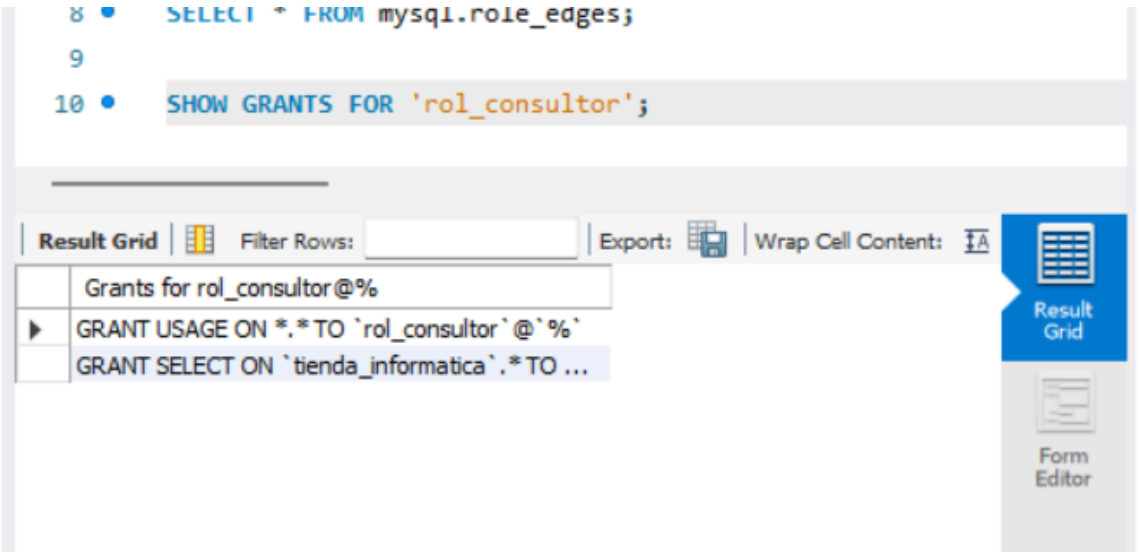
HOST	USER	DEFAULT_ROLE_HOST	DEFAULT_ROLE_USER
NULL	NULL	NULL	NULL

- **`GRANT SELECT ON tienda_informatica.* TO 'rol_consultor';`**: Le da permiso exclusivamente de consulta (SELECT / leer) sobre todas las tablas de la base de datos `tienda_informatica` al rol.

3. Asignación y rol por defecto: Asigna el rol rol_consultor al usuario auditor_externo y configúralo explícitamente como rol por defecto para su sesión de trabajo.

```
GRANT 'rol_consultor' TO 'auditor_externo'@'%';
 SET DEFAULT ROLE 'rol_consultor' FOR 'auditor_externo'@'%';
```





SHOW GRANTS FOR 'rol_consultor';

- **`GRANT .. TO ...` y `SET DEFAULT ROLE`**: Asigna ese rol al usuario `auditor_externo` y hace que se active automáticamente cada vez que inicie sesión.

T3. SEGURIDAD MEDIANTE VISTAS Y PRIVILEGIOS GRANULARES.

1. Creación de la vista de enmascaramiento: Crea una vista llamada v_tecnicos_activos que muestre únicamente el identificador, el nombre y la especialidad de los técnicos, ocultando por completo la columna de salarios.

- **rear la tabla 'tecnicos' (si no existe) con sus campos, incluido el salario confidencial**

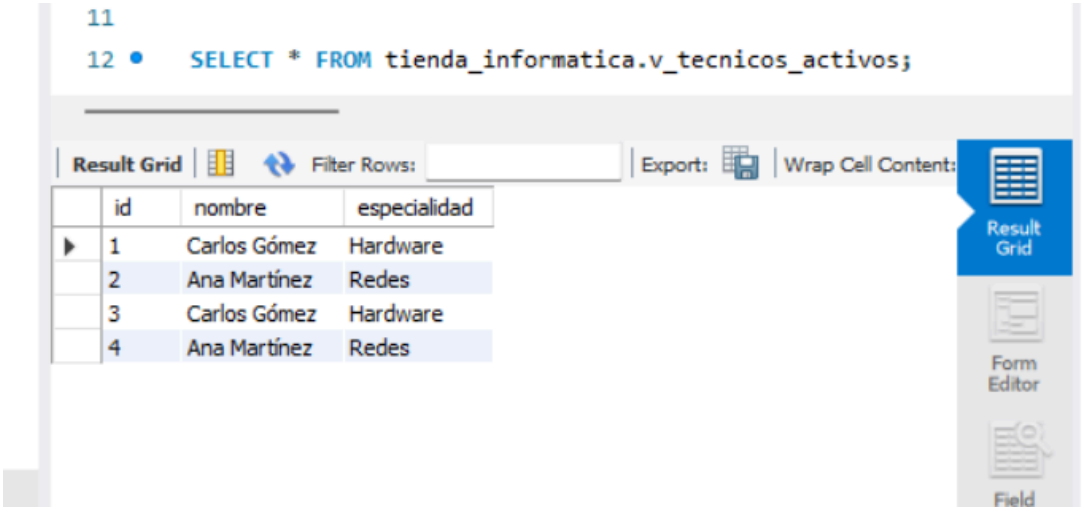
```
CREATE TABLE IF NOT EXISTS tecnicos (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100),
   especialidad VARCHAR(100),
   salario DECIMAL(10,2)
);
```

- **Insertar algunos datos de ejemplo para poder probar la vista.**

```
INSERT INTO tecnicos (nombre, especialidad, salario) VALUES
('Carlos Gómez', 'Hardware', 1800.00),
('Ana Martínez', 'Redes', 2100.00);
```

Comando para comprobar la vista para ver los datos de la tabla.

```
SELECT * FROM tienda_informatica.v_tecnicos_activos;
```



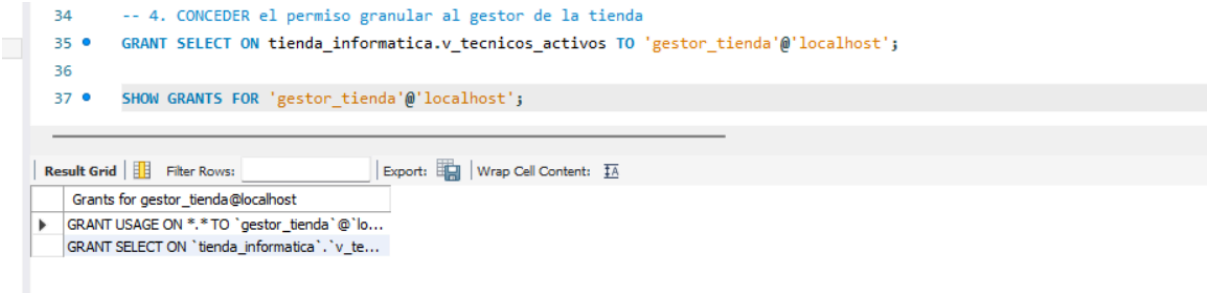
2. Privilegio granular: Concede al usuario gestor_tienda permisos exclusivos de consulta sobre dicha vista v_tecnicos_activos.

- se hace con este siguiente cuando.
  USE tienda_informatica;

-- Conceder al usuario 'gestor_tienda' permisos exclusivos de consulta sobre la vista
GRANT SELECT ON tienda_informatica.v_tecnicos_activos TO 'gestor_tienda'@'localhost';



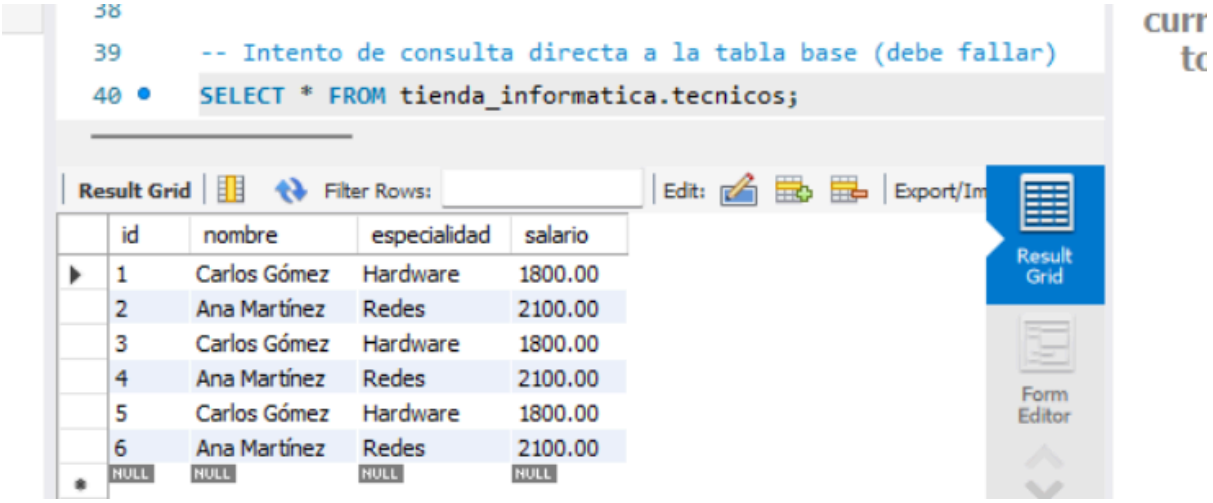
verificarlo con el comando
SHOW GRANTS FOR 'gestor_tienda'@'localhost';



3. Prueba de denegación: Comprueba y documenta qué ocurre si el usuario gestor_tienda intenta consultar directamente la tabla base tecnicos.

- se hace con este siguiente comando.

SELECT * FROM tienda_informatica.tecnicos;



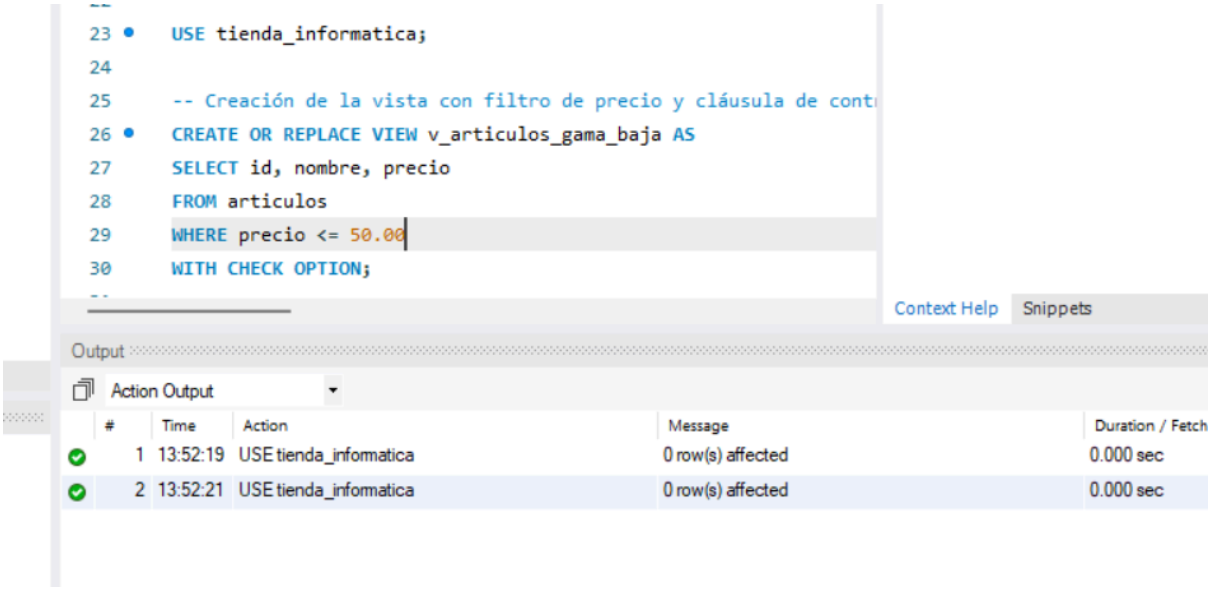
T4 CONSISTENCIA Y CONTROL CON (WITH CHECK OPTION).

1. Vista con restricción de dominio: Crea una vista llamada v_articulos_gama_baja basada en la tabla articulos que filtre aquellos productos cuyo precio sea inferior o igual a 50.00 euros (precio <= 50.00), incorporando obligatoriamente la cláusula WITH CHECK OPTION.

Se hace con este siguiente comando.

```
-- 1. Crear la tabla de artículos si no existe
CREATE TABLE IF NOT EXISTS articulos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    precio DECIMAL(10,2)
);

-- 2. Crear la vista de gama baja con la restricción WITH CHECK OPTION
CREATE OR REPLACE VIEW v_articulos_gama_baja AS
SELECT id, nombre, precio
FROM articulos
WHERE precio <= 50.00
WITH CHECK OPTION;
```

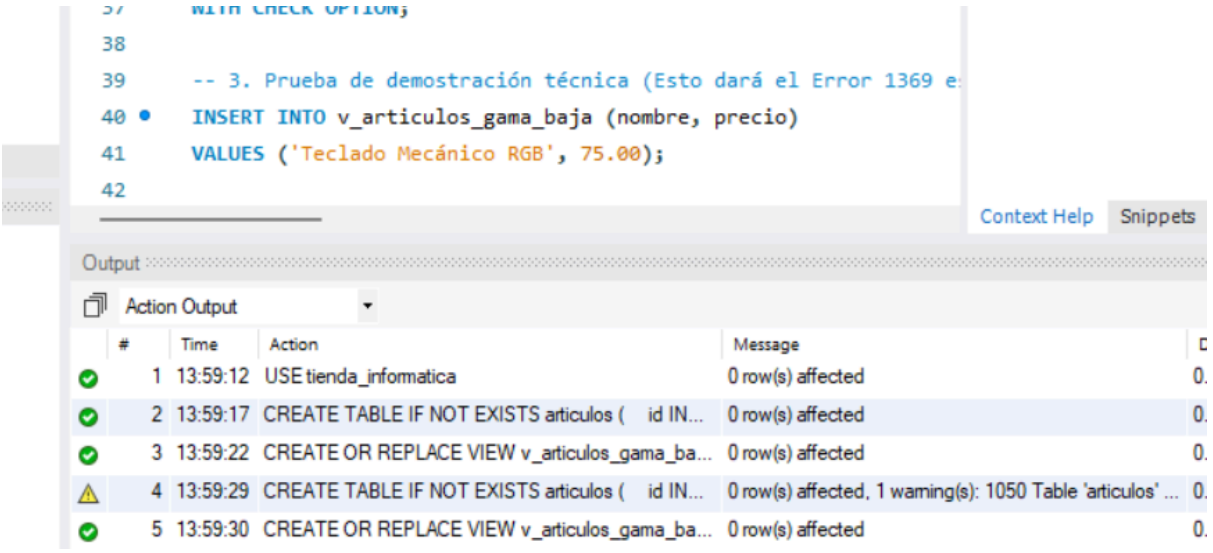


verificarlo con el siguiente comando

2. Demostración técnica: Razona y demuestra técnicamente qué sucede si el administrador o un usuario con permisos intenta realizar una inserción de un artículo con un precio de 75.00 euros a través de dicha vista.

3. Prueba de demostración técnica (Esto dará el Error 1369 esperado)

INSERT INTO v_articulos_gama_baja (nombre, precio)
VALUES ('Teclado Mecánico RGB', 75.00);



T5 AUDITORIA, RESOLUCIÓN DE INCIDENCIAS Y REFLEXIÓN CRÍTICA.

1. Auditoría de privilegios efectivos: Imagina que el auditor externo reporta que no puede ver los cambios recientes en la base de datos. Escribe la sentencia SQL que utilizarías para comprobar los privilegios exactos y los roles activos que tiene asignados en este momento el usuario auditor_externo desde el servidor.

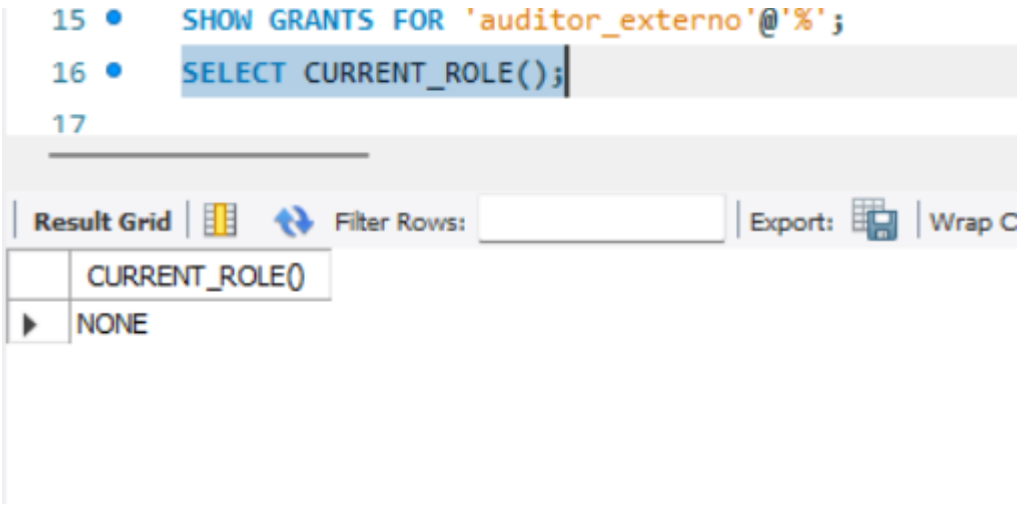
- Muestra todos los privilegios directos concedidos y los roles asignados al usuario

SHOW GRANTS FOR 'auditor_externo'@'%';



- Muestra los roles que están activos en la sesión actual del usuario

SELECT CURRENT_ROLE();



Para qué sirve: `SHOW GRANTS` muestra los permisos exactos que tiene asignados, y `SELECT CURRENT_ROLE();` verifica si el rol de solo lectura está activo o si necesita ser activado (`SET ROLE`).

2. Reflexión sobre el Principio de Mínimo Privilegio: Explica razonadamente por qué en un entorno de producción nunca se debe otorgar el permiso ALL PRIVILEGES a los usuarios finales de una aplicación web o de gestión, y qué relación directa guarda esta mala práctica con las inyecciones SQL o brechas de seguridad.

- Principio de Mínimo Privilegio significa dar a cada usuario o aplicación **solo los** permisos indispensables para trabajar, ni uno más.
- Por qué no usar `ALL PRIVILEGES`: Si la aplicación sufre una inyección SQL (SQLi), un atacante con permisos totales podrá destruir o robar toda la base de datos.
- Beneficio: Con permisos limitados, el daño se frena en seco y se evita un desastre total.

3. Análisis de comportamiento del CHECK OPTION: Si un analista junior elimina la cláusula WITH CHECK OPTION en la v_articulos_gama_baja pensando que "mejora el rendimiento de las consultas", ¿qué brecha lógica o inconsistencia de datos podría introducir en la tabla base articulos al insertar registros mediante la vista? Justifica tu respuesta.

`CHECK OPTION`

- Qué pasa si se quita: La vista deja de bloquear los precios altos en las inserciones.
- El problema: Se colarían productos prohibidos en la tabla principal y desaparecerían al mirar la vista (porque el filtro los oculta), rompiendo la integridad de los datos.